

INITIATION À LA PROGRAMMATION

avec le langage Python

De zéro à l'autonomie

Formateur :

DJOSSOU S. Rodrigue & Conceptio L. M. HOUESSOU

Chapitre 1 — Concepts Fondamentaux

1.1 Qu'est-ce qu'un langage de programmation ?

Un langage de programmation est une notation conventionnelle permettant de formuler des algorithmes et de produire des programmes informatiques. Comme une langue vivante, il possède :

- Un alphabet (les caractères autorisés)
- Un vocabulaire (les mots-clés réservés)
- Des règles de grammaire et de syntaxe (comment assembler les instructions)

À retenir

Si on ne respecte pas la syntaxe d'un langage, l'ordinateur ne comprend pas le programme et renvoie une erreur. C'est comme parler à quelqu'un dans une langue qu'il ne connaît pas !

1.2 Qu'est-ce qu'un algorithme ?

Un algorithme est une suite finie et non ambiguë d'instructions permettant de résoudre un problème. Le mot vient du mathématicien perse Al-Khwârizmî (IXe siècle).

Exemples d'algorithmes de la vie courante :

- Une recette de cuisine : des instructions pour réaliser un plat
- Un itinéraire routier : tourner à gauche, continuer tout droit...
- Une chorégraphie : une suite d'étapes pour réaliser une performance
- La méthode de division posée apprise à l'école
- L'algorithme d'Euclide : pour trouver le PGCD de deux entiers

Important

Il peut exister plusieurs algorithmes pour résoudre un même problème.

On choisira le plus efficace selon des critères comme :

- La vitesse d'exécution
- La consommation de mémoire
- La précision du résultat

1.3 Qu'est-ce qu'un programme informatique ?

Un programme informatique est un ensemble d'opérations destinées à être exécutées par un ordinateur. C'est la traduction en langage machine d'une collection d'algorithmes.

Les programmes que vous utilisez tous les jours (navigateur web, tableur, application mobile) sont des programmes informatiques écrits par des développeurs.

1.4 Les paradigmes de programmation

Un paradigme est une façon d'aborder et d'organiser la programmation. Voici les principaux :

Paradigme	Principe	Exemple
Impératif	Modifier successivement des variables via des instructions, boucles, conditions	C, Pascal
Fonctionnel	Composer des fonctions mathématiques pures sans effets de bord	Haskell, Lisp
Orienté objet	Organiser le code en objets regroupant données et comportements	Java, C++
Déclaratif	Décrire ce qu'on veut obtenir, pas comment l'obtenir	SQL, HTML

Dans ce cours

Nous utilisons principalement le paradigme IMPÉRATIF, le plus intuitif pour débiter. Python le supporte nativement, et c'est ainsi que fonctionnent la majorité des programmes que vous écrierez.

1.5 Niveaux des langages de programmation

Les langages se distinguent selon leur proximité avec le langage machine :

Niveau	Description	Exemples
Haut niveau	Syntaxe proche de l'anglais, facile à lire et écrire. Le compilateur/interpréteur traduit vers la machine.	Python, Java, JavaScript
Assembleur	Représentation symbolique du langage machine. Chaque instruction correspond à une opération processeur.	ASM x86, ARM
Langage machine	Suite de 0 et de 1 directement compris par le processeur. Incompréhensible pour un humain.	Code binaire

Chapitre 2 — Le Langage Python

2.1 Historique et popularité

Python a été créé en 1990 par le Néerlandais Guido van Rossum. C'est aujourd'hui l'un des langages les plus populaires au monde, utilisé dans des domaines très variés :

- Intelligence Artificielle et Machine Learning
- Science des données (Data Science)
- Développement web (Django, Flask)
- Automatisation et scripting
- Analyse d'images et vision par ordinateur
- Recherche scientifique

✓ Saviez-vous ?

Le cœur du site YouTube est codé en Python !

Des géants comme Google, Instagram, Spotify, Netflix utilisent Python.

Python est le langage N°1 recommandé pour débiter la programmation.

2.2 Pourquoi apprendre Python en premier ?

Python présente plusieurs avantages pour les débutants :

- Syntaxe claire et lisible, proche de l'anglais naturel
- Moins de règles formelles que d'autres langages (pas de points-virgules obligatoires, etc.)
- Interprété : on voit le résultat immédiatement sans compilation
- Communauté immense et documentation abondante
- Bibliothèques disponibles pour presque tous les domaines

2.3 Comment exécuter du Python ?

Il existe deux modes d'exécution :

Mode interactif (REPL) :

On tape une instruction, Python l'exécute immédiatement et affiche le résultat. Idéal pour tester rapidement.

Terminal — Mode interactif

```
$ python3
>>> 2 + 3
5
>>> print('Bonjour !')
Bonjour !
```

Mode script :

On écrit toutes les instructions dans un fichier `.py`, puis on l'exécute d'un coup. C'est la méthode pour créer de vrais programmes.

Terminal — Exécution d'un script

```
$ python3 mon_programme.py
```

Chapitre 3 — Les Notions de Base

3.1 L'instruction

Une instruction est un ordre donné à l'ordinateur. En Python, chaque ligne est généralement une instruction. Voici la plus célèbre, qui affiche un message à l'écran :

Python — Ma première instruction

```
print("Bonjour, monde !")

# Résultat :
Bonjour, monde !
```

La fonction print()

print() est une fonction intégrée qui affiche du texte à l'écran.

On met le texte entre guillemets (simples ' ' ou doubles " ").

On peut afficher plusieurs éléments séparés par des virgules :

```
print("J'ai", 25, "ans")
```

3.2 Les variables

Une variable est un espace de stockage nommé où l'on conserve une valeur. Imaginez une boîte avec une étiquette : l'étiquette, c'est le nom de la variable ; ce qu'il y a dedans, c'est sa valeur.

Créer et utiliser une variable :

Python — Variables

```
# On crée une variable 'age' et on lui assigne la valeur 20
age = 20

# On affiche sa valeur
print('Mon âge : ', age)
# Résultat : Mon âge : 20

# On peut modifier la valeur
age = 21
print('Mon âge : ', age)
# Résultat : Mon âge : 21
```

Tester l'égalité de deux valeurs (attention : == et non = !):

Python — Affectation vs Comparaison

```
age = 20
age == 20    # True   (est-ce que age vaut 20 ?)
age == 42    # False  (est-ce que age vaut 42 ?)
```

```
# ATTENTION à ne pas confondre :
# age = 20    → affecte 20 à la variable age
# age == 20   → teste si age est égal à 20
```

Règles de nommage des variables

Le nom d'une variable doit respecter ces règles :

- Commencer par une lettre ou un underscore _ (jamais un chiffre)
- Ne contenir que des lettres, chiffres et underscores (pas d'espace, ni de tiret)
- Ne pas être un mot-clé réservé de Python
- Python est sensible à la casse : age, Age et AGE sont trois variables différentes !

Python — Nommage des variables

```
# Noms VALIDES :
ma_variable = 10
prenom2 = 'Alice'
_compteur = 0

# Noms INVALIDES :
2variable = 5      # commence par un chiffre → ERREUR
mon-nom = 'Bob'    # contient un tiret → ERREUR
for = 3            # mot-clé réservé → ERREUR
```

⚠ Mots-clés réservés

Ne jamais utiliser ces mots comme noms de variables :

and, as, assert, break, class, continue, def, del, elif, else,
except, finally, for, from, global, if, import, in, is, lambda,
not, or, pass, print, raise, return, try, while, with, yield,
False, None, True

3.3 Les types de données

Chaque valeur en Python a un type qui détermine sa nature et les opérations qu'on peut effectuer dessus. Voici les types primitifs fondamentaux :

Type	Nom complet	Exemples	Description
int	Entier	0, 42, -7, 1000	Nombre entier (positif ou négatif)
float	Flottant	3.14, -0.5, 2.0	Nombre décimal (à virgule)
bool	Booléen	True, False	Valeur de vérité (vrai ou faux)
str	Chaîne	'Bonjour', "Alice"	Texte entre guillemets
complex	Complexe	2+3j, 1j	Nombre complexe (avancé)

Python — La fonction type()

```
# Vérifier le type d'une valeur avec type()
print(type(42))           # <class 'int'>
print(type(3.14))         # <class 'float'>
print(type(True))         # <class 'bool'>
print(type('Bonjour'))   # <class 'str'>
print(type(2+3j))         # <class 'complex'>
```

3.4 Le type booléen (bool) en détail

Le type bool ne peut prendre que deux valeurs : True (vrai) ou False (faux). Il est fondamental pour les conditions et les tests.

Conversion en booléen :

- Tout nombre vaut True, sauf 0 qui vaut False
- Toute chaîne de caractères vaut True, sauf la chaîne vide "" qui vaut False

Python — Conversions booléennes

```
print(bool(0))           # False ← le seul entier qui est faux
print(bool(1))           # True
print(bool(-42))         # True ← tout nombre non nul est vrai
print(bool(''))          # False ← chaîne vide = faux
print(bool('Oui'))       # True

# PIÈGE COURANT :
print(bool('False'))     # True ← la chaîne 'False' est non vide !
```

⚠ Piège à éviter !

`bool('False')` vaut True car la chaîne 'False' est non vide.

C'est une erreur courante chez les débutants !

Pour tester une valeur booléenne, utilisez directement True/False, pas leur représentation en chaîne.

Chapitre 4 — Les Opérateurs

4.1 Opérateurs de comparaison

Ces opérateurs comparent deux valeurs et renvoient un résultat booléen (True ou False).

Opérateur	Signification	Exemple	Résultat
==	Égal à	5 == 5	True
!=	Différent de	5 != 3	True
<	Inférieur à	3 < 5	True
>	Supérieur à	7 > 10	False
<=	Inférieur ou égal à	5 <= 5	True
>=	Supérieur ou égal à	4 >= 7	False

Python — Opérateurs de comparaison

```
x = 15
print(x == 15)    # True
print(x != 15)    # False
print(x > 10)     # True
print(x <= 15)    # True

# On peut aussi comparer des chaînes (ordre alphabétique)
print('a' < 'b')  # True
print('Z' < 'a')  # True (les majuscules ont des codes plus petits)
```

4.2 Opérateurs logiques

Ces opérateurs combinent plusieurs conditions booléennes. Ils sont au nombre de trois : and, or, not.

Opérateur	Signification	Résultat
A and B	Et : vrai si A ET B sont tous les deux vrais	True seulement si A=True et B=True
A or B	Ou : vrai si au moins l'un est vrai	False seulement si A=False et B=False
not A	Non : inverse la valeur booléenne	True si A=False, False si A=True

Table de vérité complète :

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True

False	False	False	False	True
-------	-------	-------	-------	------

Python — Opérateurs logiques

```
x = 12

# Est-ce que x est pair ET strictement inférieur à 20 ?
print(x % 2 == 0 and x < 20)      # True

# Est-ce que x est négatif OU supérieur à 10 ?
print(x < 0 or x > 10)            # True

# Inverse d'un test
print(not (x == 12))              # False
```

✓ Évaluation paresseuse (lazy evaluation)

Python est 'paresseux' : dans 'A and B', si A est déjà False, B n'est même pas évalué (le résultat ne peut qu'être False).
De même dans 'A or B', si A est True, B n'est pas évalué.
C'est important à savoir pour éviter des erreurs subtiles.

4.3 Opérations arithmétiques

Python propose toutes les opérations mathématiques courantes :

Opération	Symbole	Exemple	Résultat
Addition	+	7 + 3	10
Soustraction	-	7 - 3	4
Multiplication	*	7 * 3	21
Division réelle	/	7 / 2	3.5
Division entière	//	7 // 2	3
Modulo (reste)	%	7 % 2	1
Puissance	**	2 ** 8	256
Valeur absolue	abs()	abs(-5)	5

Python — Opérations arithmétiques

```
print(17 // 5)      # 3   (quotient de la division euclidienne)
print(17 % 5)       # 2   (reste de la division euclidienne)

# Vérification : 17 = 5 × 3 + 2  ✓

print(2 ** 10)      # 1024
print(round(3.14159, 2)) # 3.14  (arrondi à 2 décimales)
```

```
# Conversions de type
print(int(3.9))    # 3    (tronque, ne arrondit pas !)
print(float(5))    # 5.0
print(str(42))     # '42'
```

4.4 Opérateurs avec assignation

Ces raccourcis permettent de modifier une variable en place. Très utilisés pour incrémenter des compteurs :

Raccourci	Équivalent	Exemple
<code>x += y</code>	<code>x = x + y</code>	<code>i += 1</code> (incrmente i)
<code>x -= y</code>	<code>x = x - y</code>	<code>total -= 5</code>
<code>x *= y</code>	<code>x = x * y</code>	<code>double *= 2</code>
<code>x /= y</code>	<code>x = x / y</code>	<code>prix /= 2</code>
<code>x //= y</code>	<code>x = x // y</code>	<code>n //= 10</code>
<code>x %= y</code>	<code>x = x % y</code>	<code>reste %= 7</code>
<code>x **= y</code>	<code>x = x ** y</code>	<code>carre **= 2</code>

Python — Opérateurs avec assignation

```
compteur = 0
compteur += 1    # compteur vaut maintenant 1
compteur += 1    # compteur vaut maintenant 2
compteur *= 3    # compteur vaut maintenant 6
print(compteur)  # 6
```

4.5 Attention aux flottants !

Les nombres à virgule (float) sont stockés en binaire avec une précision limitée. Cela peut causer des surprises :

Python — Précision des flottants

```
print(0.1 + 0.2)          # 0.30000000000000004    (pas 0.3 !)
print(0.1 + 0.2 == 0.3)   # False    ← piège classique !
print(0.2 + 0.2 == 0.4)   # True     ← ça marche ici par chance

# 1 - 1/3 - 1/3 - 1/3 devrait valoir 0 :
print(1 - 1/3 - 1/3 - 1/3) # 1.11e-16    (presque 0 !)
print(1 - (1/3 + 1/3 + 1/3)) # 0.0        (l'ordre change tout)
```

⚠ Règle d'or avec les flottants

Ne JAMAIS tester l'égalité exacte de deux flottants avec `==`

Utilisez plutôt un test d'approximation :

`abs(a - b) < 1e-9` (si la différence est négligeable)

Ou la fonction `math.isclose(a, b)` du module `math`.

Chapitre 5 — Les Modules

5.1 Qu'est-ce qu'un module ?

Un module est un fichier Python qui contient des fonctions, variables et classes prêtes à l'emploi. Python est livré avec une bibliothèque standard très riche, et des milliers de modules supplémentaires peuvent être installés.

Exemples de modules utiles :

- `math` : fonctions mathématiques (`cos`, `sin`, `sqrt`, `log`, `exp`...)
- `random` : génération de nombres aléatoires
- `os` : interaction avec le système d'exploitation
- `datetime` : manipulation des dates et heures
- `matplotlib` : création de graphiques

5.2 Comment importer un module ?

Il existe trois façons d'importer un module. Chacune a ses avantages :

Python — Importation de modules

```
# Méthode 1 : import complet (accès via le nom du module)
import math
print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.141592653589793

# Méthode 2 : import de tout (attention aux conflits de noms !)
from math import *
print(sqrt(16))           # 4.0 (sans le préfixe math.)

# Méthode 3 : import sélectif (RECOMMANDÉ pour les débutants)
from math import sqrt, cos, pi
print(sqrt(25))           # 5.0
print(cos(0))             # 1.0
print(pi)                 # 3.141592653589793
```

✓ Bonne pratique

Préférez la méthode 3 (import sélectif) :

```
from math import sqrt, log
```

Elle est plus claire (on sait exactement ce qu'on utilise)

et évite les conflits de noms avec vos propres variables.

Si on oublie d'importer, Python renvoie une erreur :

Python — Erreur sans import

```
cos(0)      # NameError: name 'cos' is not defined
```

```
# Solution : importer d'abord !  
from math import cos  
cos(0)    # 1.0
```

5.3 Principales fonctions du module math

Fonction	Description	Exemple
<code>math.sqrt(x)</code>	Racine carrée de x	<code>sqrt(9) → 3.0</code>
<code>math.pow(x, y)</code>	x à la puissance y	<code>pow(2, 8) → 256.0</code>
<code>math.exp(x)</code>	Exponentielle e^x	<code>exp(1) → 2.718...</code>
<code>math.log(x)</code>	Logarithme naturel (base e)	<code>log(1) → 0.0</code>
<code>math.log10(x)</code>	Logarithme base 10	<code>log10(100) → 2.0</code>
<code>math.cos(x)</code>	Cosinus (x en radians)	<code>cos(0) → 1.0</code>
<code>math.sin(x)</code>	Sinus (x en radians)	<code>sin(pi/2) → 1.0</code>
<code>math.floor(x)</code>	Arrondi vers le bas	<code>floor(3.9) → 3</code>
<code>math.ceil(x)</code>	Arrondi vers le haut	<code>ceil(3.1) → 4</code>
<code>math.pi</code>	La constante π	3.14159265...
<code>math.e</code>	La constante e	2.71828182...

Chapitre 6 — Les Structures Conditionnelles

6.1 Principe des conditions

Les structures conditionnelles permettent d'exécuter des instructions seulement si certaines conditions sont remplies. C'est la base de toute décision dans un programme.

Python utilise les mots-clés : `if` (si), `elif` (sinon si), `else` (sinon).

⚠ Règle fondamentale : l'indentation

En Python, l'indentation (les espaces en début de ligne) est OBLIGATOIRE et a un sens précis. Elle délimite les blocs d'instructions.

Convention : utiliser 4 espaces (ou 1 tabulation) par niveau.

Une mauvaise indentation = une erreur ou un comportement inattendu !

6.2 `if` — Le test simple

Forme : si la condition est vraie, on exécute le bloc ; sinon, on passe à la suite sans rien faire.

Syntaxe — `if`

```
if condition:
    # bloc exécuté si condition est True
    instruction1
    instruction2
# suite du programme (toujours exécutée)
```

Python — Exemple `if`

```
temperature = 35

if temperature > 30:
    print('Il fait chaud !')
    print('Pensez à vous hydrater.')

print('Bonne journée !')    # toujours affiché
```

6.3 `if / else` — Test avec alternative

Forme : si la condition est vraie, on exécute le premier bloc ; sinon, on exécute le bloc `else`.

Syntaxe — `if / else`

```
if condition:
    # exécuté si condition est True
    instructions_si_vrai
else:
    # exécuté si condition est False
```

```
instructions_si_faux
```

Python — Exemple if / else

```
age = int(input('Quel est votre âge ? '))

if age >= 18:
    print('Vous êtes majeur(e).')
else:
    print('Vous êtes mineur(e).')
```

6.4 if / elif / else — Tests multiples

Forme : on teste plusieurs conditions à la suite. Dès qu'une condition est vraie, son bloc est exécuté et on sort du test. Si aucune ne l'est, le bloc else est exécuté.

Syntaxe — if / elif / else

```
if condition1:
    instructions1
elif condition2:
    instructions2
elif condition3:
    instructions3
else:
    instructions_par_defaut
```

Python — Système de notes

```
note = int(input('Entrez votre note sur 20 : '))

if note >= 16:
    print('Très Bien')
elif note >= 14:
    print('Bien')
elif note >= 12:
    print('Assez Bien')
elif note >= 10:
    print('Passable')
else:
    print('Insuffisant')
```

Remarques importantes

1. Dès qu'une condition elif est vraie, les suivantes sont ignorées.
2. On peut avoir autant de elif qu'on veut.
3. Le else est optionnel (mais il ne peut y en avoir qu'un).
4. Les deux-points : en fin de ligne sont OBLIGATOIRES.

6.5 La fonction input()

La fonction `input()` permet de demander une saisie à l'utilisateur. Elle retourne toujours une chaîne de caractères (`str`), qu'il faut convertir si on attend un nombre.

Python — La fonction `input()`

```
# Saisie d'un texte
nom = input('Entrez votre nom : ')
print('Bonjour,', nom, '!')

# Saisie d'un entier (conversion obligatoire !)
age = int(input('Entrez votre âge : '))
print('Dans 10 ans, vous aurez', age + 10, 'ans.')

# Saisie d'un flottant
prix = float(input('Prix en FCFA : '))
```

Chapitre 7 — Les Boucles

7.1 Principe des boucles

Une boucle permet de répéter un bloc d'instructions plusieurs fois. Il existe deux types de boucles en Python :

- La boucle `for` : boucle inconditionnelle, on connaît le nombre d'itérations à l'avance
- La boucle `while` : boucle conditionnelle, on répète tant qu'une condition est vraie

7.2 La boucle `for`

La boucle `for` parcourt une séquence (liste, chaîne, `range`...) et exécute le bloc pour chaque élément.

Syntaxe — `for`

```
for variable in sequence:
    # instructions répétées pour chaque élément
    instructions
```

L'objet `range()`

`range()` génère une séquence d'entiers. C'est l'outil principal pour faire des boucles `for` un certain nombre de fois.

Forme	Description	Résultat
<code>range(n)</code>	De 0 à n-1	<code>range(5)</code> → 0,1,2,3,4
<code>range(a, b)</code>	De a à b-1	<code>range(2,6)</code> → 2,3,4,5
<code>range(a, b, h)</code>	De a à b-1, pas de h	<code>range(0,10,2)</code> → 0,2,4,6,8
<code>range(a, b, -1)</code>	De a à b+1, décroissant	<code>range(5,0,-1)</code> → 5,4,3,2,1

Python — Exemples de boucle `for`

```
# Afficher les nombres de 1 à 5
for i in range(1, 6):
    print(i)
# Résultat : 1 2 3 4 5

# Compter à rebours
for i in range(5, 0, -1):
    print(i)
# Résultat : 5 4 3 2 1

# Somme des entiers de 1 à 100
total = 0
for k in range(1, 101):
    total += k
print('Somme =', total)    # 5050
```

⚠ La valeur de fin est EXCLUE

`range(1, 6)` génère 1, 2, 3, 4, 5 — mais PAS 6 !

Pour aller de 1 à N inclus, on écrit `range(1, N+1)`.

Exemple : pour afficher de 1 à 10, on écrit `range(1, 11)`.

Python — Test d'appartenance à un range

```
# Tester si un nombre appartient à un range
r = range(0, 100, 2)    # nombres pairs de 0 à 98
print(50 in r)          # True
print(51 in r)          # False (51 est impair)
print(100 in r)         # False (100 est exclu)
```

break et continue

Deux mots-clés permettent de contrôler le comportement d'une boucle :

- **break** : quitte immédiatement la boucle
- **continue** : passe à l'itération suivante en sautant le reste du bloc

Python — break et continue

```
# break : on s'arrête dès qu'on trouve 3
for n in range(0, 6):
    if n == 3:
        print('Trouvé 3 ! On arrête.')
        break
    print(n)
# Résultat : 0  1  2  Trouvé 3 ! On arrête.

# continue : on saute les nombres pairs
for n in range(0, 6):
    if n % 2 == 0:
        continue    # on ignore les pairs
    print(n)         # seulement les impairs
# Résultat : 1  3  5
```

7.3 La boucle while

La boucle **while** répète un bloc d'instructions tant qu'une condition reste vraie. On l'utilise quand on ne sait pas à l'avance combien d'itérations seront nécessaires.

Syntaxe — while

```
while condition:
    # instructions répétées tant que condition est True
    instructions
    # IMPORTANT : modifier la condition pour éviter la boucle infinie !
```

Python — Exemples de boucle while

```
# Compter de 1 à 5 avec while
```

```

i = 1
while i <= 5:
    print(i)
    i += 1      # ← ESSENTIEL : sans ça, boucle infinie !

# Demander un nombre positif
n = int(input('Entrez un nombre positif : '))
while n <= 0:
    print('Erreur ! Le nombre doit être positif.')
    n = int(input('Réessayez : '))
print('Merci ! Vous avez entré : ', n)

```

⚠ Danger : la boucle infinie !

Si la condition d'un while ne devient jamais False, le programme tourne indéfiniment. C'est une boucle infinie !

Pour l'arrêter dans le terminal : Ctrl + C

Toujours vérifier que la condition évoluera vers False.

Exemple de boucle infinie (NE PAS faire) :

i = 1

while i > 0: ← i est toujours > 0 !

i += 1

Python — Cas d'usage typique du while

```

# Exemple : trouver le plus petit n tel que 1+2+...+n >= 1000
somme = 0
n = 0
while somme < 1000:
    n += 1
    somme += n

print(f'Le plus petit n est : {n}')      # 45
print(f'La somme 1+2+...+{n} = {somme}') # 1035

# Note : on NE PEUT PAS faire ça avec une boucle for,
# car on ne connaît pas n à l'avance.

```

7.4 for vs while — Comment choisir ?

Situation	Boucle recommandée
Répéter N fois (N connu à l'avance)	for
Parcourir une liste, une chaîne, un range	for
Répéter jusqu'à ce qu'une condition soit remplie	while

Attendre une saisie valide de l'utilisateur	while
Algorithmes de convergence (on ne sait pas combien d'étapes)	while

Chapitre 8 — Les Fonctions

8.1 Pourquoi les fonctions ?

Imaginez que vous devez calculer la factorielle à trois endroits différents de votre programme. Sans fonction, vous copiez-collez le même code 3 fois. Avec une fonction, vous l'écrivez une seule fois et l'appellez quand vous en avez besoin.

Les fonctions permettent de :

- Éviter la répétition de code (principe DRY : Don't Repeat Yourself)
- Décomposer un problème complexe en sous-problèmes plus simples
- Rendre le code plus lisible et maintenable
- Réutiliser du code dans d'autres programmes

8.2 Définir et appeler une fonction

Syntaxe — Définition d'une fonction

```
def nom_de_la_fonction(parametre1, parametre2):  
    # corps de la fonction  
    instructions  
    return resultat    # optionnel
```

Python — Première fonction

```
# Définition de la fonction  
def dire_bonjour(prenom):  
    message = 'Bonjour, ' + prenom + ' !'  
    print(message)  
  
# Appels de la fonction  
dire_bonjour('Alice')    # Bonjour, Alice !  
dire_bonjour('Bob')      # Bonjour, Bob !  
dire_bonjour('Rodrigue') # Bonjour, Rodrigue !
```

8.3 Fonction avec valeur de retour (return)

Une fonction peut calculer une valeur et la retourner avec `return`. Le résultat peut alors être utilisé dans une expression ou stocké dans une variable.

Python — Fonction avec return

```
def carre(x):  
    return x ** 2  
  
def cube(x):  
    return x ** 3
```

```
# Utilisation
print(carre(5))      # 25
print(cube(3))       # 27

# On peut utiliser le résultat dans une expression
hypotenuse = (carre(3) + carre(4)) ** 0.5
print(hypotenuse)    # 5.0

# Ou le stocker dans une variable
resultat = cube(4)
print(resultat)      # 64
```

return arrête la fonction

Dès que Python rencontre `return`, la fonction s'arrête et renvoie la valeur indiquée. Le code qui suit `return` dans la même fonction n'est jamais exécuté.

Une fonction sans `return` renvoie `None` implicitement.

8.4 Paramètres et arguments

Une fonction peut prendre plusieurs paramètres. L'ordre est important lors de l'appel, sauf si on nomme les arguments :

Python — Ordre des paramètres

```
def rectangle_info(longueur, largeur):
    perimetre = 2 * (longueur + largeur)
    aire = longueur * largeur
    print(f'Périmètre : {perimetre}')
    print(f'Aire      : {aire}')

# Appel classique (ordre important !)
rectangle_info(10, 5)

# Appel avec arguments nommés (ordre libre !)
rectangle_info(largeur=5, longueur=10)
```

8.5 Paramètres avec valeurs par défaut

On peut donner une valeur par défaut à certains paramètres. Si l'appelant ne les fournit pas, la valeur par défaut est utilisée.

Python — Paramètres par défaut

```
def saluer(prenom, langue='fr'):
    if langue == 'fr':
        print('Bonjour,', prenom, '!')
    elif langue == 'en':
        print('Hello,', prenom, '!')
```

```

    else:
        print('Salut,', prenom, '!')

saluer('Alice')          # Bonjour, Alice !  (langue='fr' par défaut)
saluer('Bob', 'en')      # Hello, Bob !
saluer('Carlos', langue='es')  # Salut, Carlos !

```

⚠ Règle d'ordre

Les paramètres SANS valeur par défaut doivent toujours être placés AVANT les paramètres AVEC valeur par défaut.

def f(x, y, z=0, t=1): # OK

def f(x, y=0, z, t=1): # ERREUR → z n'a pas de défaut mais vient après y qui en a un

8.6 Les fonctions lambda

Python offre une syntaxe compacte pour définir de petites fonctions anonymes sur une seule ligne :

Syntaxe — lambda

```
nom = lambda arguments : expression
```

Python — Fonctions lambda

```

# Fonction classique
def carre(x):
    return x ** 2

# Équivalent en lambda
carre = lambda x : x ** 2
print(carre(5))    # 25

# Lambda avec plusieurs arguments
somme = lambda x, y : x + y
produit = lambda x, y : x * y

print(somme(3, 4))    # 7
print(produit(3, 4))  # 12

# Lambda utile avec des fonctions comme sorted()
mots = ['banane', 'kiwi', 'pomme', 'ananas']
tries = sorted(mots, key=lambda m : len(m))  # tri par longueur
print(tries)    # ['kiwi', 'pomme', 'banane', 'ananas']

```

8.7 Portée des variables : locale vs globale

La portée d'une variable désigne l'endroit où elle est accessible dans le code.

Type	Définie où ?	Accessible où ?
Locale	À l'intérieur d'une fonction	Seulement dans cette fonction
Globale	En dehors de toute fonction	Partout dans le fichier (lecture seule dans les fonctions)

Python — Variables locales et globales

```
# Variable locale : inaccessible en dehors
def ma_fonction():
    x = 42    # x est locale
    print('Dans la fonction, x =', x)

ma_fonction()    # Dans la fonction, x = 42
# print(x)       # NameError : x n'existe pas ici !

# Variable globale : accessible en lecture dans une fonction
total = 100

def afficher_total():
    print('Total :', total)    # lecture OK

afficher_total()    # Total : 100
```

Python — Le mot-clé global

```
# Modifier une variable globale : utiliser le mot-clé global
compteur = 0

def incrementer():
    global compteur    # on indique qu'on veut modifier la globale
    compteur += 1

incrementer()
incrementer()
incrementer()
print(compteur)    # 3
```

✓ Conseil de bonne pratique

L'utilisation de 'global' est à éviter autant que possible.
Préférez passer les valeurs en paramètre et les récupérer
via return. Le code sera plus clair et plus facile à déboguer.

Chapitre 9 — Exercices Pratiques

Exercice 1 — Factorielle

La factorielle de n (notée $n!$) est définie par : $n! = 1 \times 2 \times 3 \times \dots \times n$, avec $0! = 1$.

Travail demandé :

1. Écrire une fonction `factorielle_for(n)` qui calcule $n!$ avec une boucle `for`.
2. Écrire une fonction `factorielle_while(n)` qui calcule $n!$ avec une boucle `while`.
3. Vérifier vos résultats : $5! = 120$, $10! = 3628800$.

Solution — Exercice 1

```
# Solution avec for
def factorielle_for(n):
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat

# Solution avec while
def factorielle_while(n):
    resultat = 1
    i = 1
    while i <= n:
        resultat *= i
        i += 1
    return resultat

# Tests
print(factorielle_for(5))      # 120
print(factorielle_while(10))   # 3628800
```

Exercice 2 — FooBar

Pour chaque entier de 1 à 50 :

- Afficher "foobar" s'il est multiple de 3 ET de 5
- Afficher "foo" s'il est seulement multiple de 3
- Afficher "bar" s'il est seulement multiple de 5
- Afficher le nombre sinon

Solution — Exercice 2

```
for n in range(1, 51):
    if n % 3 == 0 and n % 5 == 0:
        print('foobar')
    elif n % 3 == 0:
        print('foo')
    elif n % 5 == 0:
        print('bar')
    else:
        print(n)
```

```

elif n % 5 == 0:
    print('bar')
else:
    print(n)

# Extrait du résultat attendu :
# 1, 2, foo, 4, bar, foo, 7, 8, foo, bar, ..., foobar, ...

```

Exercice 3 — Nombre de façons d'obtenir 20 avec 5 dés

Combien y a-t-il de façons différentes d'obtenir une somme de 20 en lançant 5 dés à 6 faces (faces numérotées 1 à 6) ?

Solution — Exercice 3

```

compteur = 0

for d1 in range(1, 7):
    for d2 in range(1, 7):
        for d3 in range(1, 7):
            for d4 in range(1, 7):
                for d5 in range(1, 7):
                    if d1 + d2 + d3 + d4 + d5 == 20:
                        compteur += 1

print('Nombre de façons :', compteur) # 651

```

Exercice 4 — La conjecture de Syracuse

Définissez une suite (x_n) avec x_0 quelconque. Pour chaque n :

- Si x_n est pair : $x_{n+1} = x_n / 2$
- Si x_n est impair : $x_{n+1} = 3 \times x_n + 1$

La conjecture affirme que quelle que soit la valeur de départ $x_0 > 0$, on atteint toujours 1.

Travail demandé : écrire une fonction `syracuse(x0)` qui affiche la suite et le nombre d'itérations nécessaires pour atteindre 1.

Solution — Exercice 4

```

def syracuse(x0):
    x = x0
    n = 0
    print(x, end=', ')
    while x != 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = 3 * x + 1
        n += 1

```

```
        print(x, end=', ')
    print()
    print(f'On atteint 1 après {n} itération(s) pour x0={x0}')

syracuse(27)    # 111 itérations
syracuse(42)    # 8 itérations
```

Exercice 5 — Mini-calculatrice

Écrire un programme qui :

4. Demande deux nombres à l'utilisateur
5. Demande l'opération souhaitée (+, -, *, /)
6. Affiche le résultat
7. Gère l'erreur de division par zéro

Solution — Exercice 5

```
def calculatrice():
    a = float(input('Premier nombre : '))
    b = float(input('Deuxième nombre : '))
    op = input('Opération (+, -, *, /) : ')

    if op == '+':
        print('Résultat :', a + b)
    elif op == '-':
        print('Résultat :', a - b)
    elif op == '*':
        print('Résultat :', a * b)
    elif op == '/':
        if b == 0:
            print('Erreur : division par zéro impossible !')
        else:
            print('Résultat :', a / b)
    else:
        print('Opération inconnue !')

calculatrice()
```

Récapitulatif — Ce que vous savez faire

Notion	Syntaxe clé	À retenir
Variable	<code>x = 5</code>	Stocke une valeur, modifiable
Types	<code>int, float, str, bool</code>	Chaque valeur a un type
Comparaison	<code>==, !=, <, >, <=, >=</code>	Retourne True ou False
Logique	<code>and, or, not</code>	Combine des booléens
Arithmétique	<code>+, -, *, /, //, %, **</code>	<code>//</code> = quotient, <code>%</code> = reste
Condition	<code>if / elif / else</code>	Indentation obligatoire
Boucle <code>for</code>	<code>for i in range(...):</code>	Nombre d'itérations connu
Boucle <code>while</code>	<code>while condition:</code>	Condition évaluée à chaque tour
Fonction	<code>def f(x): ... return ...</code>	Réutilise du code
Module	<code>from math import sqrt</code>	Réutilise des bibliothèques

✓ Pour aller plus loin

Les prochaines notions à explorer :

- Les listes et les tuples (collections de valeurs)
- Les dictionnaires (association clé → valeur)
- Les chaînes de caractères et leurs méthodes
- La programmation orientée objet (classes et objets)
- Les fichiers (lire et écrire dans des fichiers)
- Les bibliothèques `numpy`, `pandas`, `matplotlib` pour la data

Bonne programmation !